

Chefe e funcionário no React

Oct 8, 2026 · @Jefferson

"Chefe" e "funcionário" não são tipos diferentes de componente. Todo componente é uma função que devolve tela. O papel depende só da posição: quem usa outro componente é o chefe dele; quem é usado é o funcionário.

O que o chefe pode fazer

- **Guardar dados** num `useState`. Exemplo: o `App` guarda a lista `livros`.
- **Mudar esses dados** com a função `set`. Exemplo: `setLivros`.
- **Decidir quais funcionários aparecem** e quantos. Exemplo: o `map` cria um `Livro` para cada livro.
- **Entregar dados** aos funcionários pelas props. Exemplo: `titulo={livro.titulo}`.
- **Entregar funções** aos funcionários pelas props, para eles poderem pedir mudanças. Exemplo: `marcarLido={marcarLido}`.

O que o funcionário pode fazer

- **Ler** o que recebeu pelas props. Exemplo: `props.titulo`.
- **Mostrar** isso na tela, do jeito que quiser. Exemplo: o título num `<h2>`.
- **Chamar** as funções que recebeu, para pedir uma mudança ao chefe. Exemplo: `props.marcarLido(props.titulo, true)`.
- **Ter o seu próprio `useState`**, para coisas que só interessam a ele. Exemplo: no começo, o `lido` ficava no `Livro`, antes de o chefe precisar saber.

O que o funcionário não pode fazer

- **Mudar as props diretamente.** Escrever `props.lido = true` não funciona. Ele precisa **pedir** ao chefe.
- **Enxergar os dados do chefe** que não recebeu pelas props.
- **Enxergar os dados de outro funcionário.** Um `Livro` não sabe nada sobre os outros livros.

E o chefe do grupo?

Ele faz as duas coisas ao mesmo tempo:

- Para o chefe principal, ele é **funcionário**: recebe props e chama funções.
- Para os seus funcionários, ele é **chefe**: entrega props e funções.

Exemplo: uma `ListaDeLivros` receberia `livros` e `marcarLido` do `App` e repassaria a cada `Livro`.

A regra que resume tudo

Quem guarda um dado é o único que pode mudá-lo. Os outros só podem ler ou pedir.